

Module 3 — Maths for ML

Revolut's topic 1 asks for "concepts like **bias and variance**". This module is the machinery every model shares, whatever its name: what a model is, how it learns (loss, gradient descent), why it fails (under- and overfitting — the bias–variance trade-off), how you keep it honest (validation, leakage), and how you measure it (the metrics). Module 4 then walks through the individual models — and each one will be "this machinery, with a different shape".

THE ONE IDEA OF THIS MODULE

A model is judged on data it has never seen. Everything here — the split, the regularisation, the metrics — exists to answer one question honestly: *how well will this work on tomorrow's customers?*

Modules 1–2 gave you the pieces: a model's output is a conditional probability (M1); bias and variance are the two kinds of error (M2); a test set is a sample, so its metrics have confidence intervals (M2). This module assembles them.

c1 What a model is

A **model** is a function with adjustable numbers inside: $\hat{y} = f(x; w)$. You feed it **features** x (what you know about a case), it returns a **prediction** $\hat{y}$ of the **target** y (what you want to know). The adjustable numbers w are the **parameters** (weights). **Training** = using past cases where y is known to find good values of w .

TERM	MEANING	CREDIT-DEFAULT EXAMPLE
Features (x)	inputs known at decision time	credit limit, age, last 6 months of repayment status
Target / label (y)	what you predict	defaulted next month: 1 / 0
Parameters (w)	learned from data	the weight on "months late"
Hyperparameters	chosen by you, before training	how deep a tree may grow; the strength of regularisation
Training / inference	learning w from history / applying the model to a new case	fit on last year's loans / score today's applicant

The types of learning

- **Supervised** — you have labels. **Regression**: the target is a number (LTV, amount, days to repay). **Classification**: the target is a category (fraud / not; default / not). Most business ML is supervised classification.
- **Unsupervised** — no labels; find structure. **Clustering** (customer segments), **dimensionality reduction**, **anomaly detection**.
- **Self-supervised** — the data labels itself: hide part of the input and predict it. That is how language models and **PRAGMA** are pre-trained (mask events in a customer's history, learn to reconstruct them).
- **Reinforcement learning** — learn by acting and receiving rewards. One line is enough.

30 SECONDS — "WHAT IS SUPERVISED MACHINE LEARNING?"

"Learning a function from examples where the answer is known. I have historical cases with features — what I knew at the time — and a label — what happened. Training finds the parameters that make the model's predictions match the labels as closely as possible, and then I apply it to new cases where the label isn't known yet. If the label is a number it's regression; if it's a category, like default or no default, it's classification. Without labels it's unsupervised learning — clustering, for instance."

EXERCISES C1

1. Regression, classification, or unsupervised? (a) Predict how much a customer will spend next month. (b) Group merchants by behaviour with no predefined categories. (c) Decide whether a login is an account takeover. (d) Predict which of five products a customer buys next.
2. For a "will this dispute be resolved in the customer's favour?" model, name three plausible features and the target. Name one feature that would be tempting but invalid, and why.
3. Parameter or hyperparameter: the weights of a logistic regression; the number of trees in a forest; the learning rate; the split thresholds inside a tree.

c2 How a model learns: loss and gradient descent

The loss — one number that says how wrong the model is

LOSS	FORMULA	USED FOR	CHARACTER
MSE — mean squared error	$(1/n) \sum (y - \hat{y})^2$	regression	punishes big errors heavily (squared) → sensitive to outliers; predicts the <i>mean</i>
MAE — mean absolute error	$(1/n) \sum y - \hat{y} $	regression	every euro of error counts the same → robust to outliers; predicts the <i>median</i>
Log loss (cross-entropy)	$-(1/n) \sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	classification	punishes confident wrong answers enormously: saying 1 % when it happens costs $\log(0.01)$

Log loss is the multiplication rule from Module 1 in disguise: the probability the model assigns to all the observed outcomes is a product; take the log and it becomes a sum; minimise the negative. That's **maximum likelihood** (Module 2).

Training = finding the weights that minimise the loss

Picture the loss as a landscape: every choice of weights is a location, the loss is the altitude. Training walks downhill.

```
repeat:  $w \leftarrow w - \eta \cdot (\text{slope of the loss at } w)$   $\eta$  = learning rate
```

- The **gradient** is the slope: which direction makes the loss go up fastest. Step the other way.
- The **learning rate** η is the step size. Too small: training crawls. Too large: it overshoots and bounces or diverges.
- **Stochastic gradient descent (SGD)**: compute the slope on a small random batch instead of all the data — noisier steps, but far cheaper; it's how neural networks train. **Adam** is SGD with an automatically adapted step size.
- An **epoch** is one pass through the training data.
- Linear regression has a closed-form solution (the "normal equation") — no iterations needed — but gradient descent scales better to huge data. Trees don't use gradient descent at all; they search for the best splits.

30 SECONDS – "WHAT IS GRADIENT DESCENT?"

"It's how most models are trained. I define a loss — a number measuring how wrong the predictions are — and I want the weights that minimise it. Gradient descent starts from some weights, computes the slope of the loss, and takes a small step downhill; then repeats. The step size is the learning rate: too small and it's slow, too large and it overshoots. Stochastic gradient descent does the same on small random batches of data, which is noisier but much cheaper, and it's what neural networks use."

EXERCISES C2

1. Predictions of 10, 20, 30 against actuals of 12, 20, 60. Compute MAE and MSE. Which error dominates the MSE?
2. You predict "time to resolve a dispute". Most take 2 days; a few take 90. Would a model trained with MSE or MAE give the more typical prediction? Why?
3. A classifier says 0.99 for a case that turns out negative. Why does log loss punish that more than saying 0.6?
4. Training loss goes down, then explodes to infinity. Most likely cause?

c3 Underfitting, overfitting, and the bias–variance trade-off

This is the concept they named. It is Module 2's dartboard, applied to models.

	UNDERFITTING – HIGH BIAS	OVERFITTING – HIGH VARIANCE
What it is	model too simple to capture the pattern	model so flexible it memorises the noise in the training data
Picture	a straight line through a curve	a wiggly line through every point
Train error	high	very low
Test error	high (about the same as train)	much higher than train — the gap is the symptom
If you retrain on new data	you get nearly the same (wrong) model	you get a very different model each time
Fix	more complex model · better features · less regularisation	more data · simpler model · regularisation · fewer features · ensembling (bagging) · early stopping

expected test error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$

- **Bias:** error from wrong assumptions — the model can't represent the truth. **Variance:** error from sensitivity to the particular training sample. **Noise:** randomness in the outcome itself that no model can remove (two identical customers, one defaults and one doesn't).
- **The trade-off:** as you make a model more complex, bias falls and variance rises. Test error is U-shaped; the best model is at the bottom of the U, not at the lowest training error.
- **More data** lowers variance (the flexible model has less room to chase noise) but does nothing for bias. Same statement as in Module 2.

How you diagnose it in practice

- **Train vs. validation error.** Both high → underfitting. Train low, validation high → overfitting.
- **Learning curves** (error vs. amount of training data). If the two curves have converged at a high error, more data won't help — you need a better model or features. If there's still a gap, more data will.

REVOLUT

A fraud model shows AUC 0.99 on training data and 0.81 on last month's transactions. That gap is overfitting (or leakage — C5). A simpler model that scores 0.86 on both is the better model, even though its training number looks worse.

30 SECONDS — "EXPLAIN THE BIAS-VARIANCE TRADE-OFF"

"A model's error on new data has three parts: bias, variance and irreducible noise. Bias is error from the model being too simple — a straight line fitted to a curve; it underfits, and train and test errors are both high. Variance is error from the model being too flexible — it memorises noise in the training sample, so it does brilliantly on training data and much worse on new data; that gap is overfitting. Making a model more complex lowers bias and raises variance, so test error is U-shaped and I look for the bottom. To reduce variance: more data, regularisation, a simpler model or an ensemble. To reduce bias: a richer model or better features."

EXERCISES C3

1. Train accuracy 99 %, validation 78 %. Diagnosis? Three things you'd try.
2. Train accuracy 71 %, validation 70 %, and the business needs 85 %. Diagnosis? Will collecting ten times more data help?
3. You train the same deep tree on five different months of data and get five very different sets of rules. Which error is that, and what's the standard cure?
4. Why can't any model reach zero error on "will this customer default"? Which term of the decomposition is that?
5. In one sentence each: how does more data affect bias? variance?

c4 Regularisation — paying a little bias to buy a lot less variance

Regularisation adds a penalty for complexity to the loss, so the model prefers simple explanations unless the data insists:

$$\text{loss} = \text{error on the training data} + \lambda \times (\text{size of the weights})$$

	L2 — RIDGE	L1 — LASSO
Penalty	$\lambda \sum w_i^2$	$\lambda \sum w_i $
Effect on weights	shrinks all of them towards zero, smoothly; none become exactly zero	pushes some weights to exactly zero
Use it for	many correlated features; general stability	feature selection — a sparse, interpretable model
Both together	Elastic net: $\lambda_1 \sum w + \lambda_2 \sum w^2$	

- λ is a hyperparameter: 0 = no regularisation (may overfit); very large = all weights near zero (underfits). You choose it by validation (C5).
- **Scale the features first.** The penalty treats all weights alike, so a feature measured in cents would be penalised differently from one in euros. Standardise (subtract mean, divide by σ) before regularised linear models. Trees don't need scaling.
- Other forms of the same idea: limiting tree depth, **early stopping** (stop training when validation error starts rising), **dropout** in neural networks (randomly switch off units during training), more data, data augmentation.

- In Module 2's words: regularisation deliberately introduces a small *bias* in exchange for a large reduction in *variance*. $MSE = bias^2 + variance$ — the total can go down.

30 SECONDS – "WHAT IS REGULARISATION, AND L1 VS. L2?"

"Regularisation adds a penalty on the size of the weights to the loss, so the model only uses a feature if it really earns its place — it trades a little bias for a lot less variance and reduces overfitting. L2, or ridge, penalises squared weights: it shrinks them all smoothly and handles correlated features well. L1, or lasso, penalises absolute values: it drives some weights exactly to zero, so it does feature selection. The strength, lambda, is a hyperparameter I tune on validation data, and I standardise the features first so the penalty is fair."

EXERCISES C4

1. You have 300 features and suspect only about 20 matter. L1 or L2? Why?
2. Two features are almost copies of each other (balance in € and balance in cents ÷ 100). What does an unregularised linear model tend to do with their weights? What does L2 do?
3. You increase λ from 0 to a huge value. Describe what happens to training error and to validation error along the way.
4. Why standardise features before ridge or lasso, but not before a decision tree?

c5 Validation — keeping yourself honest

Three sets, three jobs

SET	USED TO	TYPICAL SHARE
Training	fit the parameters	60–80 %
Validation	choose hyperparameters, compare models, pick the threshold	10–20 %
Test	estimate final performance — looked at <i>once</i> , at the end	10–20 %

Why three and not two: every time you choose something because it did well on a set, you fit to that set a little. After fifty tuning decisions on the validation set, its score is optimistic. The test set has taken no part in any decision, so its number is honest. (It's the "multiple comparisons" problem of Module 2.)

Cross-validation

K-fold: split the training data into K parts (5 or 10). Train on $K - 1$, validate on the remaining one; rotate so each part is the validation set once; average the K scores. You use all the data for both training and validation and you get a sense of the score's variability. Costs K times the compute. **Stratified** K-fold keeps the class ratio the same in every fold — important for rare events.

Time matters

For anything that evolves — fraud, credit, customer behaviour — split by **time**: train on the past, validate on the more recent past, test on the most recent. A random split lets the model peek at the future and overstates performance.

Data leakage — the most common way to fool yourself

Leakage = information in the training features that would not be available at prediction time, or that comes from the target itself. The model looks superb offline and collapses in production.

KIND	EXAMPLE
Target leakage	A default model using "collections agency assigned" — that only happens <i>after</i> the default.
Time leakage	Random split on transactions: the model trains on Tuesday's fraud and "predicts" Monday's.
Group leakage	The same customer in both train and test: the model recognises the customer, not the pattern. Split by customer.
Preprocessing leakage	Computing the mean for standardisation, or selecting features, on the full data before splitting.

Red flag: a result that's too good — AUC 0.99 on a hard problem — or one feature carrying almost all the importance. Assume leakage until proven otherwise.

After deployment: drift

The world changes: **data drift** (the inputs change — new customer mix, a new market), **concept drift** (the relationship changes — fraudsters adapt). A model's test score is a snapshot. Monitor inputs and outcomes in production; retrain on a schedule or when metrics degrade.

30 SECONDS — "HOW DO YOU VALIDATE A MODEL? WHY TRAIN / VALIDATION / TEST?"

"I hold data back. The training set fits the parameters; the validation set is for every choice I make — hyperparameters, features, the threshold; and the test set is touched once at the end for an honest estimate, because anything I tuned on is optimistically biased. With limited data I use k-fold cross-validation instead of a single validation split. For anything time-dependent, like fraud or credit, I split by time, not at random, and I keep each customer on one side only. And I'm suspicious of results that look too good — that's usually leakage: a feature that wouldn't exist at prediction time."

EXERCISES C5

1. You tune 200 hyperparameter combinations and report the best validation AUC as "the model's performance". What's wrong, and which Module 2 idea is this?
2. A churn model uses the feature "number of support contacts in the 30 days before cancellation". Problem?
3. Fraud data from January to June. Describe a sensible train / validation / test split.
4. You have 2,000 labelled cases and 3 % positives. Why use stratified 5-fold cross-validation rather than one 80/20 split?
5. A model had AUC 0.88 at launch and 0.79 six months later with no code change. Two possible causes and what you'd check.
6. You standardise all features using the mean and σ of the whole dataset, then split. Is that leakage? How serious, and what's the correct order?

c6 Measuring a regression model

METRIC	FORMULA	READ IT AS
MAE	$\text{mean } y - \hat{y} $	"on average we're off by €X". Same units as y. Robust to outliers.
MSE	$\text{mean } (y - \hat{y})^2$	squared units — hard to read; it's what most models optimise.
RMSE	$\sqrt{\text{MSE}}$	same units as y; weights large errors more. $\text{RMSE} \geq \text{MAE}$ always; a big gap means a few large misses.

METRIC	FORMULA	READ IT AS
R²	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	share of the variance explained, vs. always predicting the mean. 1 = perfect, 0 = no better than the mean, negative = worse than the mean.
MAPE	$\text{mean } y - \hat{y} / y $	percentage error; breaks when y is near zero.

Squared vs. absolute error: if large misses are disproportionately costly (capacity planning, capital), use squared; if outliers are noise you don't want to chase (delays "up to 12 hours messing up the model"), use absolute, or cap / log-transform the target. Always compare against a **baseline**: predicting the mean or median for everyone.

EXERCISES C6

1. A model predicting monthly spend has MAE €40 and RMSE €160. What does the gap tell you?
2. $R^2 = 0.02$. Is the model useless? What would you want to know before deciding?
3. You predict LTV, which is heavy-tailed. Name two things you could do so that a handful of whales don't dominate training.

c7 Measuring a classifier

The confusion matrix — everything starts here

	ACTUALLY POSITIVE (DEFAULT)	ACTUALLY NEGATIVE
Predicted positive	TP — caught	FP — false alarm (Type I)
Predicted negative	FN — missed (Type II)	TN — correctly left alone

METRIC	FORMULA	QUESTION IT ANSWERS
Accuracy	$(\text{TP} + \text{TN}) / \text{all}$	share of all predictions that are right — misleading when classes are imbalanced
Precision	$\text{TP} / (\text{TP} + \text{FP})$	of those I flagged, how many were real? — $P(\text{positive} \text{flagged})$
Recall (sensitivity, TPR)	$\text{TP} / (\text{TP} + \text{FN})$	of the real ones, how many did I catch? — $P(\text{flagged} \text{positive})$
False-positive rate	$\text{FP} / (\text{FP} + \text{TN})$	of the negatives, how many did I wrongly flag?
Specificity	$\text{TN} / (\text{TN} + \text{FP}) = 1 - \text{FPR}$	of the negatives, how many did I correctly leave alone?
F1	$2 \cdot P \cdot R / (P + R)$	harmonic mean of precision and recall — one number when you need both; low if either is low

WORKED — THE ACCURACY TRAP

1,000 customers, 50 default (5 %). The model flags 100: 40 real defaulters and 60 false alarms. TP 40, FP 60, FN 10, TN 890.
 Accuracy = $930 / 1,000 = 93 \%$. Precision = $40 / 100 = 40 \%$. Recall = $40 / 50 = 80 \%$. FPR = $60 / 950 = 6.3 \%$. F1 = $2 \cdot 0.4 \cdot 0.8 / 1.2 = 0.53$.
 The model "never flag anyone" has accuracy **95 %** — higher — and catches nothing. With rare events, accuracy rewards doing nothing.

The threshold — one model, many operating points

A classifier outputs a **score** (a probability). You choose a **threshold**: flag everything above it. Lower the threshold → catch more (recall ↑) but with more false alarms (precision ↓). Raise it → the opposite. The threshold is a **business decision**: it depends on what a miss costs versus what a false alarm costs (the euro tables you built for ATO and delinquency). 0.5 is rarely the right one.

Threshold-free metrics: ROC and PR curves

- **ROC curve**: plot recall (TPR) against false-positive rate for every threshold. **AUC** (area under it): 0.5 = random, 1.0 = perfect. Interpretation worth memorising: *AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative*. It measures **ranking** quality, independent of the threshold and of the class balance.
- **PR curve**: plot precision against recall. Its baseline is the positive rate (a random model has precision = base rate). **Use PR-AUC when positives are rare**: ROC can look excellent (AUC 0.95) because the FPR's denominator is the huge negative class, while precision is still poor. PRAGMA's credit result is reported as PR-AUC for exactly this reason.
- **Calibration**: are the probabilities honest? Among cases scored 20 %, do about 20 % happen? A model can rank perfectly and be badly calibrated. It matters whenever the probability is *used as a number* — expected loss, pricing.

Imbalanced classes — the standard toolkit

- Right metrics: precision, recall, PR-AUC, cost-based — not accuracy.
- Move the threshold rather than the data, first.
- Class weights (make mistakes on the rare class cost more in the loss).
- Resampling: undersample the majority or oversample the minority (SMOTE) — on the training set only, never on validation or test.
- Stratified splits; more positive examples if you can get them.

More than two classes: a K×K confusion matrix; precision and recall per class, then averaged (macro = each class counts equally; weighted = by class size).

30 SECONDS — "HOW DO YOU EVALUATE A CLASSIFIER? IS ACCURACY A GOOD METRIC?"

"I start from the confusion matrix. Accuracy is fine when classes are balanced, but with a rare event it's misleading — if 1 % of transactions are fraud, predicting 'never fraud' is 99 % accurate. So I look at precision — of what I flagged, how much was real — and recall — of the real cases, how many I caught — and at the trade-off between them as I move the threshold, which I set from the business cost of a miss versus a false alarm. To compare models independent of threshold I use AUC — the probability a random positive outranks a random negative — and for rare positives the precision–recall curve, because ROC can look flattering there. If the probability itself is used, for pricing say, I also check calibration."

EXERCISES C7

- 10,000 transactions, 100 fraud. The model flags 300, of which 80 are fraud. Build the confusion matrix; compute accuracy, precision, recall, FPR, F1.
- Same data: what's the accuracy of "never flag"? What does that tell you about accuracy?
- You lower the threshold. What happens to TP, FP, precision, recall? Which of the four can only go one way?
- Model A: AUC 0.92. Model B: AUC 0.90. Is A always the better choice? Give one reason it might not be.
- Fraud at 0.1 %. Why might ROC-AUC = 0.97 still mean an unusable model? Which curve would show it?
- A model ranks customers perfectly by default risk but says "30 %" for a group where 10 % default. Which property is broken, and for which use does it matter?
- You oversample fraud cases to 50/50 and *then* split into train and test. What's wrong?
- For a model that auto-blocks cards, would you favour precision or recall? For one that queues cases for human review? Why?

c8 One-page cheat sheet

Model	$\hat{y} = f(x; w)$. Features in, prediction out; training finds w from labelled history. Hyperparameters are chosen, parameters are learned.
Loss	MSE (mean, outlier-sensitive) · MAE (median, robust) · log loss (classification; punishes confident mistakes).
Gradient descent	Step downhill on the loss; learning rate = step size. SGD = on random mini-batches.
Underfit / overfit	Both errors high = high bias. Train low, test high = high variance. The gap is the symptom.
Bias-variance	error = bias ² + variance + noise. Complexity ↑ → bias ↓, variance ↑. Test error is U-shaped.
Fix variance / fix bias	More data, regularise, simplify, ensemble / richer model, better features.
L2 vs. L1	Ridge shrinks all weights; lasso zeroes some (feature selection). Scale features first. λ by validation.
Train / val / test	Fit / choose / report once. K-fold when data is scarce. Split by time and by customer.
Leakage	A feature not available at prediction time. "Too good" = suspect. Then drift after launch: monitor, retrain.
Confusion matrix	Precision = TP/(TP+FP). Recall = TP/(TP+FN). FPR = FP/(FP+TN). F1 = harmonic mean.
Accuracy trap	With a 1 % event, "always no" is 99 % accurate. Use precision / recall / PR-AUC.
Threshold · AUC · calibration	Threshold = business cost decision. AUC = P(positive outranks negative). Calibration = the probabilities mean what they say.

A Answers — check only after you've written yours

C1

- (a) regression; (b) unsupervised (clustering); (c) binary classification; (d) multi-class classification.

2. Features: dispute amount, merchant's past dispute rate, customer tenure / past disputes, claim type. Target: resolved for the customer (1/0). Invalid: "refund issued" or "agent who closed the case" — known only after the outcome (leakage).
3. Parameter; hyperparameter; hyperparameter; parameter (learned from data during training).

C2

1. Errors: 2, 0, 30. $MAE = 32/3 = 10.7$. $MSE = (4 + 0 + 900)/3 = 301$ (RMSE 17.4). The single 30-point miss is 99.6 % of the MSE.
2. MAE — it targets the median (about 2 days). MSE targets the mean, which the 90-day cases drag up, so it would over-predict for typical disputes.
3. Log loss charges $-\log(\text{probability given to what happened})$. Saying 0.99 for a negative gives the truth 0.01: $-\log(0.01) = 4.6$. Saying 0.6 gives it 0.4: $-\log(0.4) = 0.92$. Confident and wrong costs five times more.
4. Learning rate too high — the steps overshoot the minimum and the loss diverges. Lower it (or check for unscaled features).

C3

1. Overfitting (high variance). Try: regularisation / simpler model (shallower trees, fewer features), more training data, cross-validated early stopping; also check for leakage.
2. Underfitting (high bias) — train and validation agree and both are too low. More data won't help; the curves have converged. Need better features or a more expressive model.
3. Variance. Cure: bagging — average many trees trained on resampled data (a random forest), or limit the depth.
4. Irreducible noise: outcomes are partly random given everything you can observe. Two identical customers can end differently.
5. More data does not reduce bias (the model still can't represent the truth). More data reduces variance.

C4

1. L1 — it drives the weights of unhelpful features to exactly zero, leaving a sparse model of the ~20 that matter.
2. Unregularised: the weights are unstable — it can put +1,000 on one and -999 on the other; tiny data changes swing them wildly (high variance). L2 splits the weight roughly evenly between them and keeps both small.
3. Training error rises steadily (the model is increasingly constrained). Validation error first falls (less overfitting), reaches a minimum, then rises (underfitting). Pick λ at the minimum.
4. The penalty compares weights across features, and a weight's size depends on the feature's units; standardising makes the penalty fair. Trees split on thresholds — any monotonic rescaling gives the same splits.

C5

1. Selection bias on the validation set: the best of 200 is partly lucky (multiple comparisons). Report the score of that chosen model on the untouched test set.
2. Leakage: "before cancellation" is defined using the outcome date. At prediction time you don't know when (or whether) they'll cancel. Use contacts in the 30 days before the *prediction* date.
3. By time: train Jan–Apr, validate May, test June (and keep each customer / card on one side if you can). Not a random split.
4. 60 positives in total; a single 20 % split has ~12 positives — its metric is extremely noisy, and an unlucky split could have 5. Stratified 5-fold keeps 12 positives in each fold and uses all 60 for evaluation; you also see the spread across folds.
5. Data drift (the population changed — new market, new product, new acquisition channel) or concept drift (fraudsters adapted; the relationship changed). Check feature distributions now vs. training, performance by

segment and by month, label delays; retrain on recent data.

6. Yes — preprocessing leakage: test-set information shaped the transformation. Usually small for means and σ , but serious for anything using the target (feature selection, target encoding). Correct order: split first; fit the scaler on train only; apply it to validation and test.

C6

1. A few very large errors: RMSE $\gg$ MAE means the error distribution has a heavy tail — most predictions are close, a handful are wildly off. Look at who they are (whales, new customers).
2. Not necessarily. It explains 2 % of the variance, which is useless for predicting an individual but can still be valuable in aggregate or as a ranking signal (a small edge over millions of decisions). Ask: what decision does it feed, and does it beat the baseline on that decision?
3. Log-transform the target; cap / winsorise it; use MAE or a robust (Huber) loss; or model "is a whale?" separately.

C7

1. TP 80, FP 220, FN 20, TN 9,680. Accuracy 97.6 %. Precision $80/300 = 26.7$ %. Recall $80/100 = 80$ %. FPR $220/9,900 = 2.2$ %. $F1 = 2 \cdot 0.267 \cdot 0.8 / 1.067 = 0.40$.
2. 99 % — higher than the model's 97.6 %, while catching nothing. Accuracy is the wrong yardstick for rare events.
3. TP $\uparrow$ (or equal), FP $\uparrow$ (or equal), recall $\uparrow$ (or equal); precision usually $\downarrow$ but can go either way. Recall can only go up as the threshold falls.
4. No. AUC averages over all thresholds; you operate at one. B may be better in the region you care about (e.g. at 1 % FPR). B may also be better calibrated, faster, simpler, more stable over time, or easier to explain to a regulator.
5. FPR's denominator is the 99.9 % of legitimate transactions, so a tiny FPR is still a mountain of false alarms: at 1 % FPR on 1 million transactions you have ~10,000 false alarms against at most 1,000 frauds — precision under 10 %. The precision–recall curve shows it.
6. Calibration. Ranking (AUC) is fine. It matters when the number is used directly — pricing, expected loss, provisioning — and doesn't matter if you only take the top N.
7. Leakage: copies (or synthetic neighbours) of the same fraud cases land in both train and test, and the test set no longer has the real 0.1 % balance. Split first; resample the training set only; evaluate on untouched data.
8. Auto-block: precision — every false positive hurts a real customer with no human to catch it. Human review queue: recall — a reviewer filters the false alarms cheaply; the costly error is a missed fraud.